

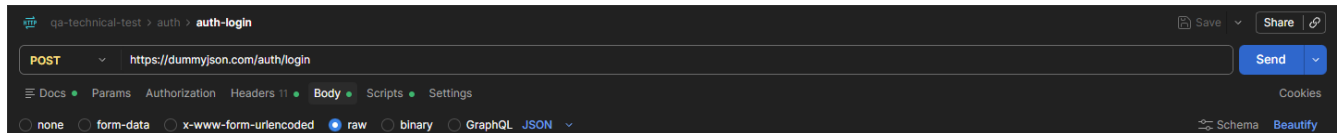
Doc: Evidencias – “qa-technical-test” realizadas por QA – Johan Daniel Ospitia Yaguara

Validación - Parte 1 – API REST (Postman)

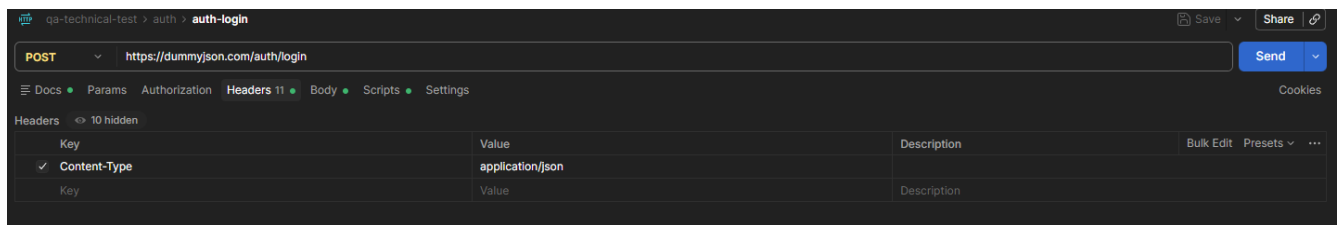
Se ingresa a aplicación POSTMAN, donde se valida servicio:

- URL | <https://dummyjson.com/>

Se valida proceso de autenticación “Auth Login” –
REQUEST: POST | <https://dummyjson.com/auth/login>

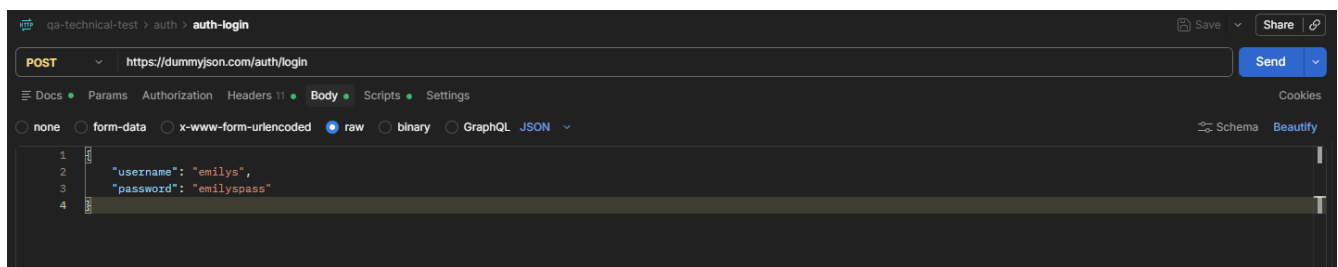


Se ingresan parámetros de cabecera: Content-Type | application/json

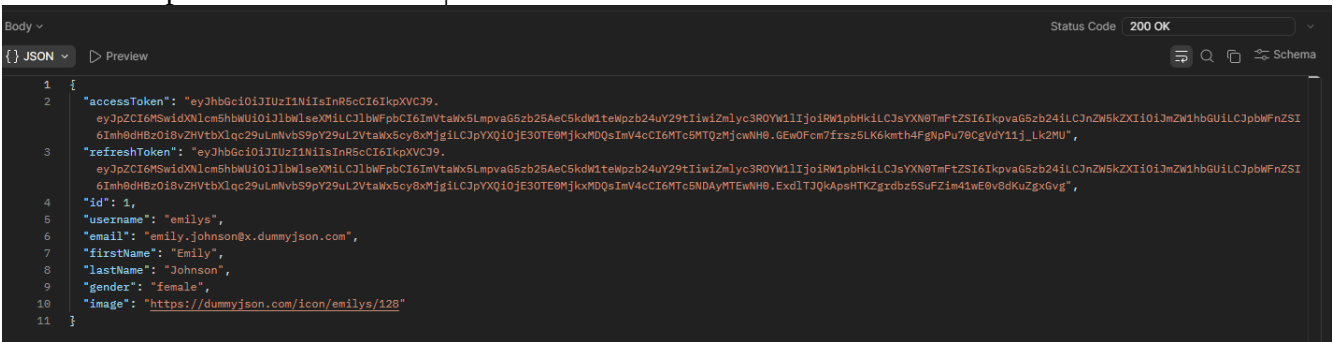


Se ingresa objeto json con datos de autenticación.

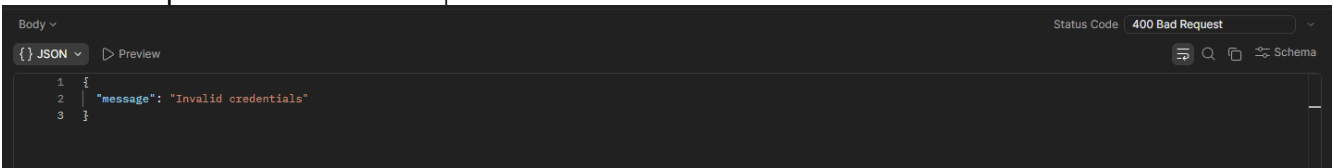
```
{  
  "username": "emilys",  
  "password": "emilypass"  
}
```



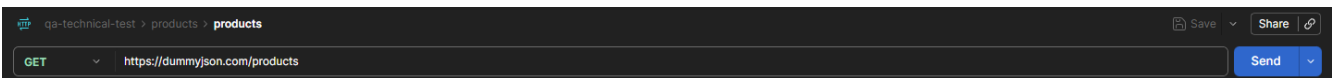
Se valida respuesta STATUS: 200 | Autenticación exitosa – retorna accessToken.



Se valida respuesta STATUS: 400 | Credenciales incorrectas

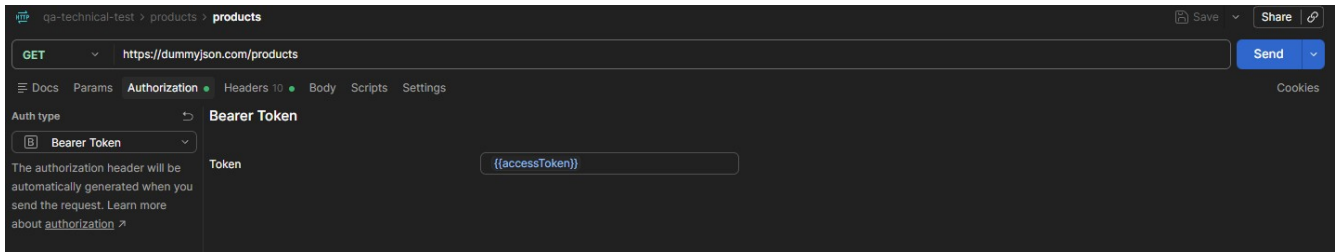


Se valida proceso de obtener lista productos “Products” –
REQUEST: GET | <https://dummyjson.com/products>

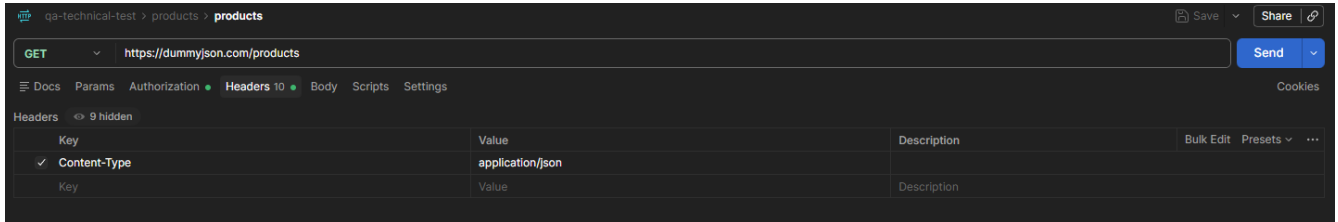


Se ingresa variable “accessToken” generada en proceso anterior.

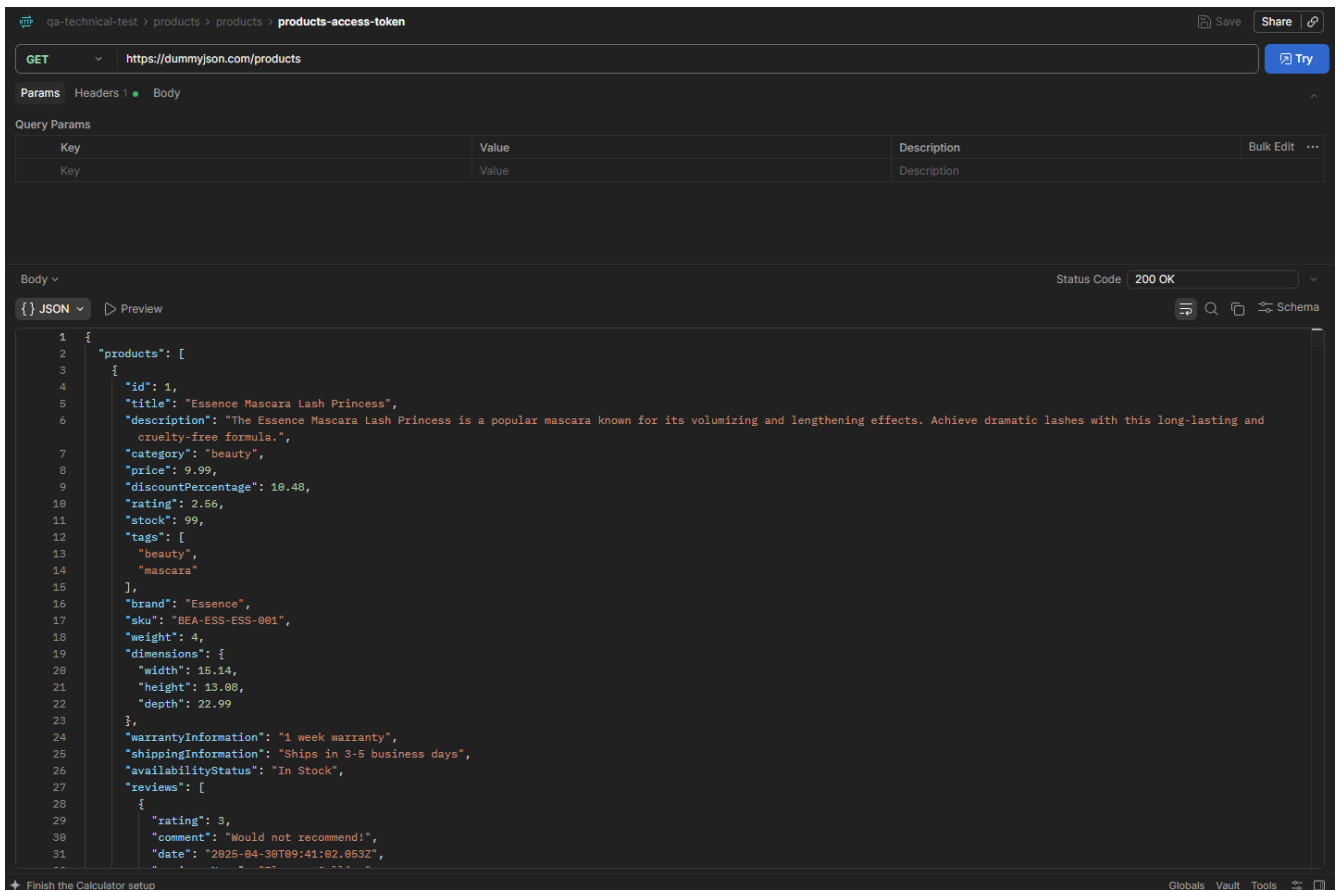




Se ingresan parámetros de cabecera: Content-Type | application/json



Se evidencia respuesta generada al consumir el servicio STATUS: 200



Validación - Parte 2 – SOAP

Se ingresa a url SOAP, donde se valida servicio:

- URL | <http://www.dneonline.com/calculator.asmx>

Calculator

The following operations are supported. For a formal definition, please review the [Service Description](#).

- [Add](#)
Adds two integers. This is a test WebService. ©DNE Online
- [Divide](#)
- [Multiply](#)
- [Subtract](#)

This web service is using <http://tempuri.org/> as its default namespace.

Recommendation: Change the default namespace before the XML Web service is made public.

Each XML Web service needs a unique namespace in order for client applications to distinguish it from other services on the Web. <http://tempuri.org/> is available for XML Web services that are under development, but published XML Web services should use a more permanent namespace.

Your XML Web service should be identified by a namespace that you control. For example, you can use your company's Internet domain name as part of the namespace. Although many XML Web service namespaces look like URLs, they need not point to actual resources on the Web. (XML Web service namespaces are URIs.)

For XML Web services creating using ASP.NET, the default namespace can be changed using the WebService attribute's Namespace property. The WebService attribute is an attribute applied to the class that contains the XML Web service methods. Below is a code example that sets the namespace to "http://microsoft.com/webservices/":

```
C#
[WebService(Namespace="http://microsoft.com/webservices/")]
public class MyWebService {
    // Implementation
}
```

Visual Basic

```
<WebService(Namespace="http://microsoft.com/webservices/")> Public Class MyWebService
    ' Implementation
End Class
```

C++

```
[WebService(Namespace="http://microsoft.com/webservices/")]
public ref class MyWebService {
    // Implementation
};
```

For more details on XML namespaces, see the W3C recommendation on [Namespaces in XML](#).

For more details on WSDL, see the [WSDL Specification](#).

For more details on URIs, see [RFC 2396](#).

Se realiza consumo del servicio “Add”

Calculator

Click [here](#) for a complete list of operations.

Add

Adds two integers. This is a test WebService. ©DNE Online

Test

The test form is only available for requests from the local machine.

SOAP 1.1

The following is a sample SOAP 1.1 request and response. The [placeholders](#) shown need to be replaced with actual values.

```
POST /calculator.asmx HTTP/1.1
Host: www.dneonline.com
Content-Type: text/xml; charset=utf-8
Content-Length: length
SOAPAction: "http://tempuri.org/Add"

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Add xmlns="http://tempuri.org/">
      <int>int</int>
      <int>int</int>
    </Add>
  </soap:Body>
</soap:Envelope>
```

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://www.dneonline.com/calculator.asmx
- Headers:** 9 hidden. Visible headers include Content-Type (text/xml; charset=utf-8), Content-Length (length), and SOAPAction ("http://tempuri.org/Add").
- Body:** The XML request body is displayed, matching the sample provided in the text.

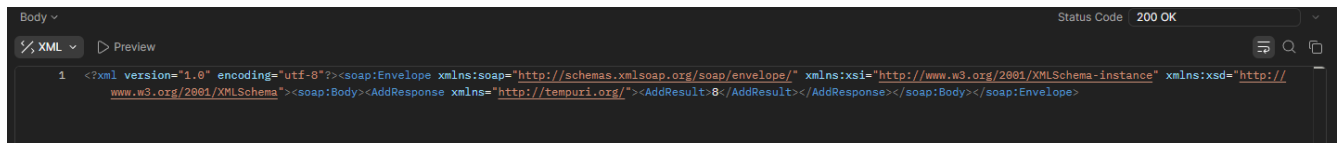
```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```

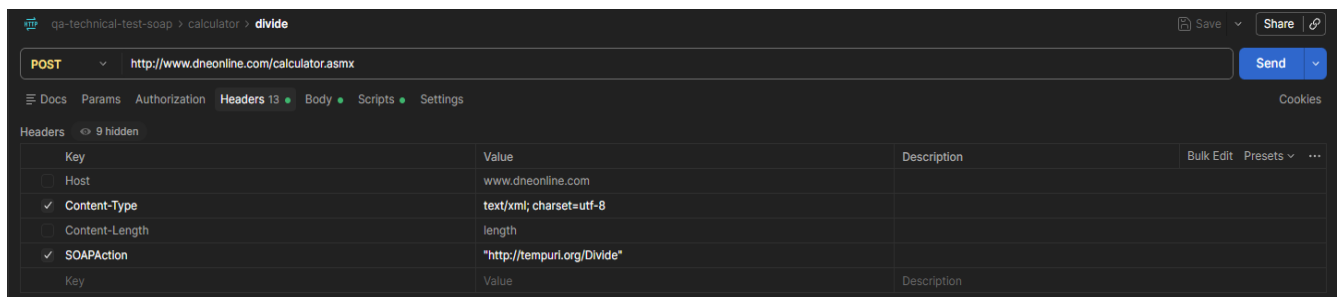
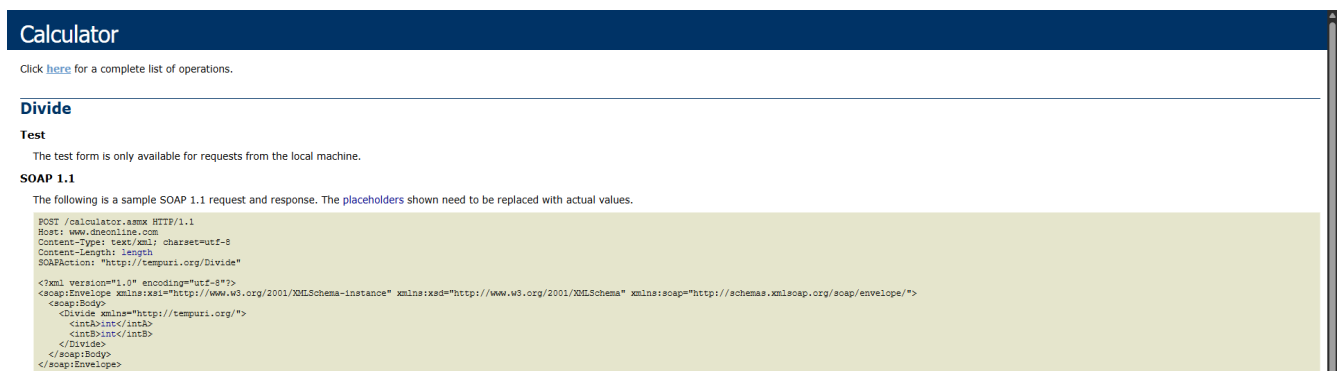
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  <soap:Body>
    <Add xmlns="http://tempuri.org/">
      <intA>5</intA>
      <intB>3</intB>
    </Add>
  </soap:Body>
</soap:Envelope>

```

Se evidencia resultado correctamente generado STATUS: 200



Se realiza consumo del servicio “Divide”



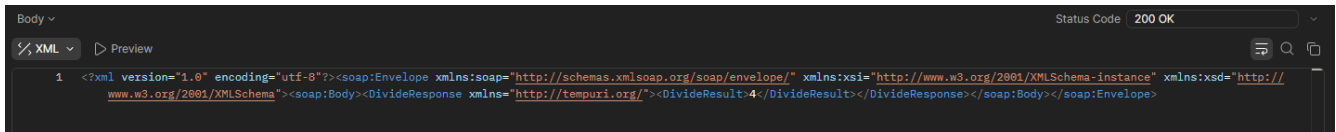
```

<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  <soap:Body>
    <Divide xmlns="http://tempuri.org/">

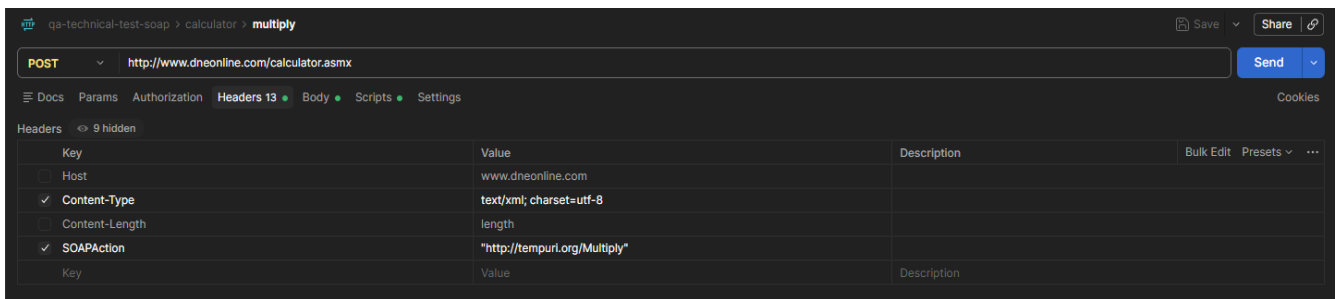
```

```
<intA>50</intA>
<intB>12</intB>
</Divide>
</soap:Body>
</soap:Envelope>
```

Se evidencia resultado correctamente generado STATUS: 200



Se realiza consumo del servicio “Multiply”

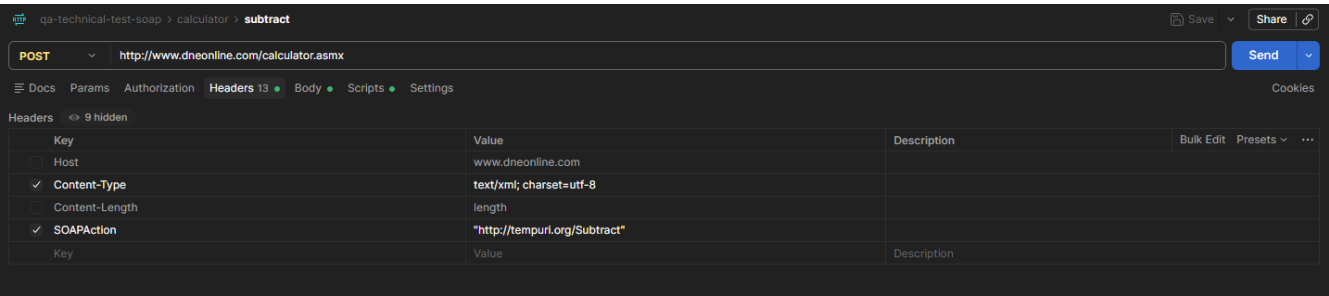
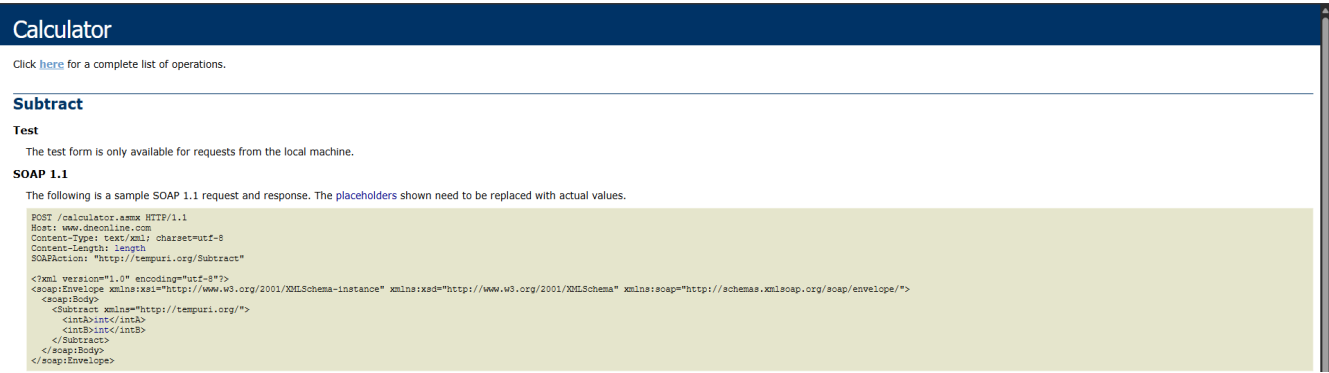


```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Multiply xmlns="http://tempuri.org/">
      <intA>2</intA>
      <intB>5</intB>
    </Multiply>
  </soap:Body>
</soap:Envelope>
```

Se evidencia resultado correctamente generado STATUS: 200

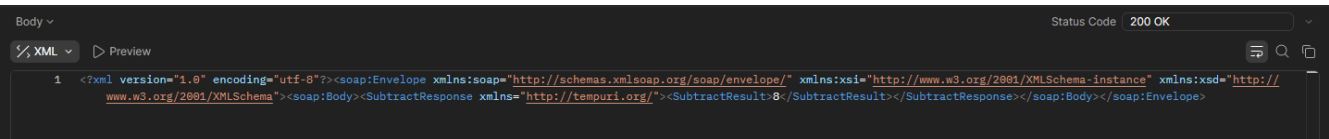


Se realiza consumo del servicio “Subtract”



```
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Subtract xmlns="http://tempuri.org/">
      <intA>10</intA>
      <intB>2</intB>
    </Subtract>
  </soap:Body>
</soap:Envelope>
```

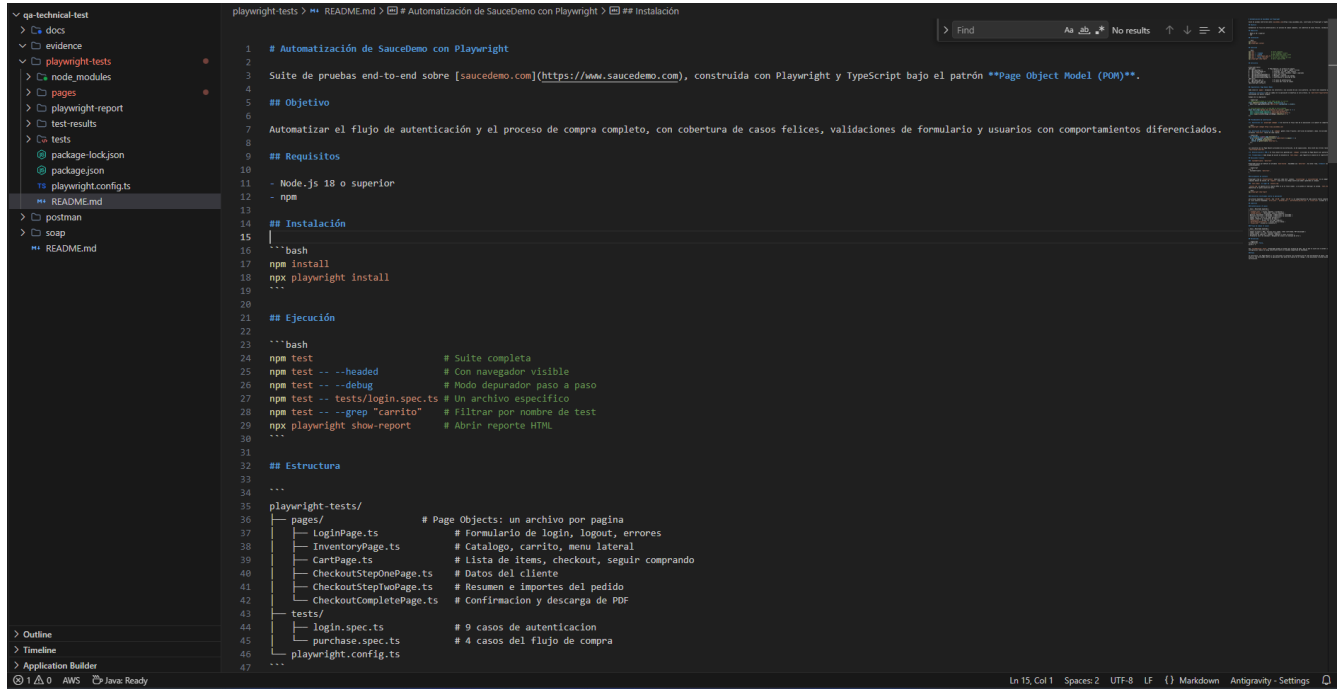
Se evidencia resultado correctamente generado STATUS: 200



Validación - Parte 3 – Playwright

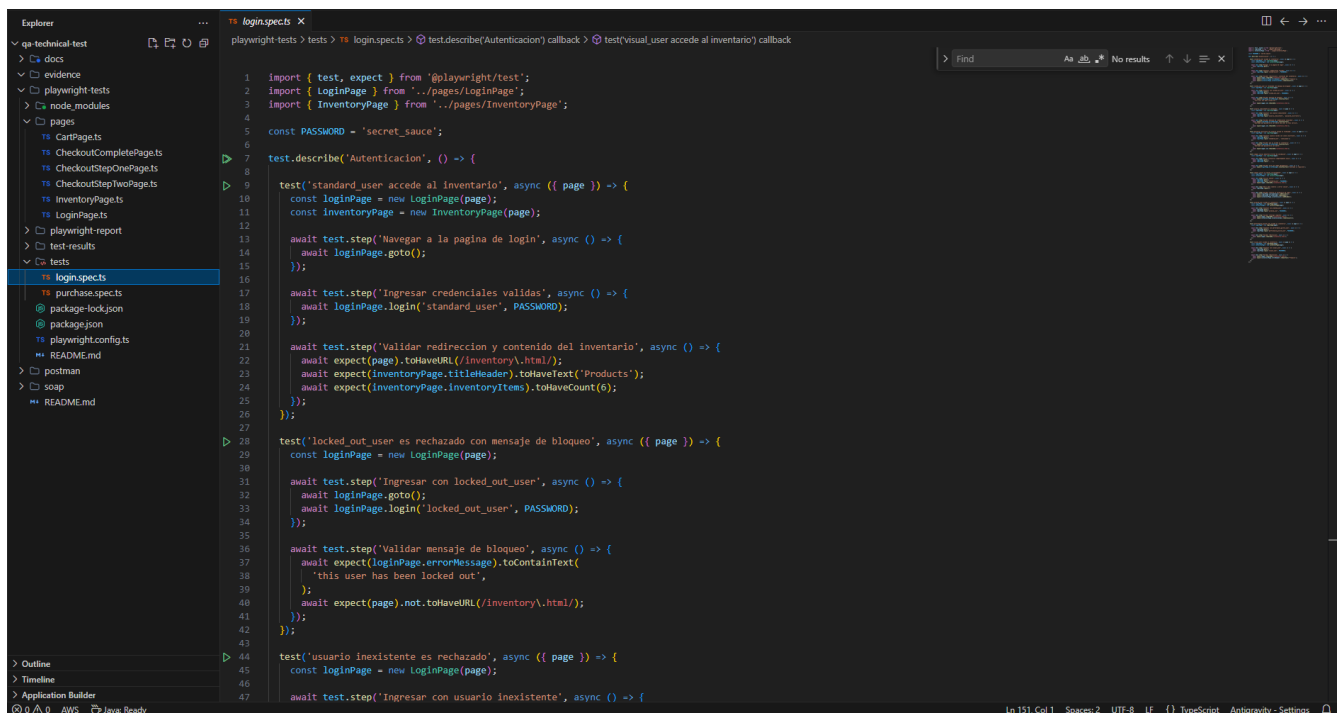
Suite de pruebas end-to-end sobre <https://www.saucedemo.com>, construida con Playwright y TypeScript bajo el patrón POM.

Se anexa documentación extra dentro del documento playwright-tests/README.md



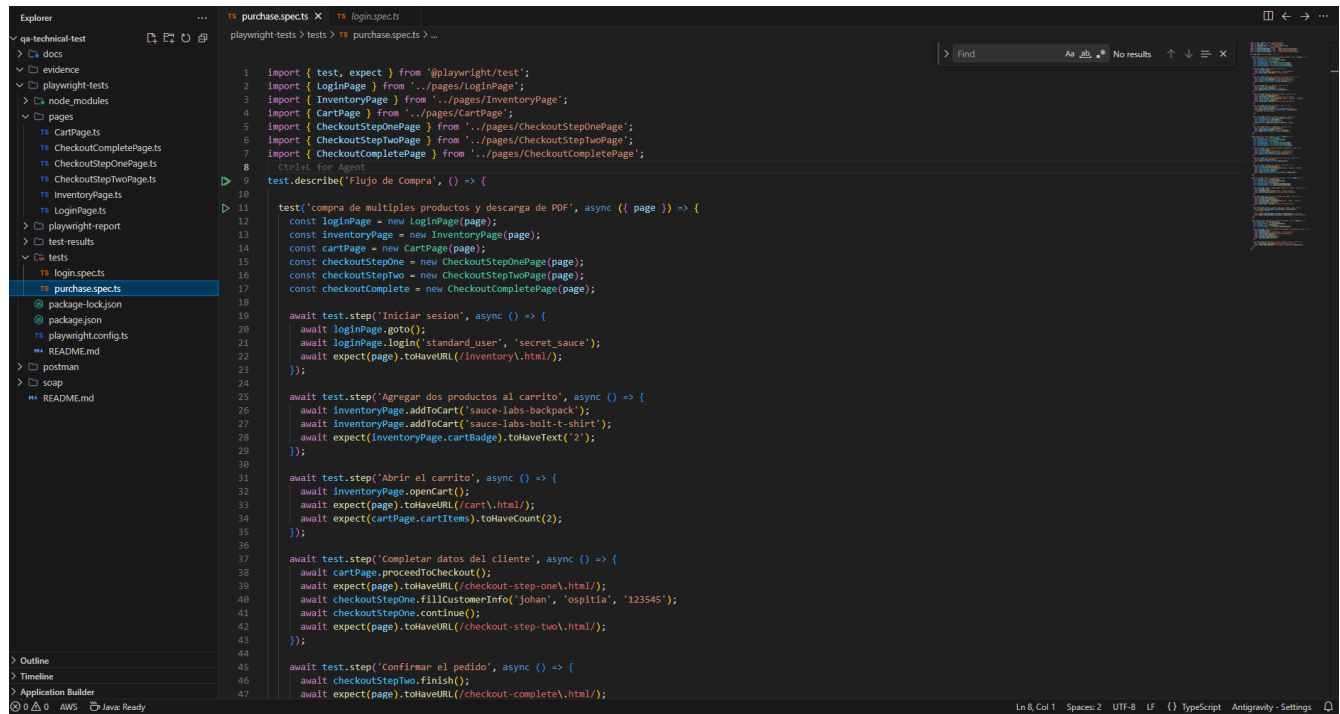
```
1 # Automatización de SauceDemo con Playwright
2
3 Suite de pruebas end-to-end sobre [saucedemo.com](https://www.saucedemo.com), construida con Playwright y TypeScript bajo el patrón **Page Object Model (POM)**.
4
5 ## Objetivo
6
7 Automatizar el flujo de autenticación y el proceso de compra completo, con cobertura de casos felices, validaciones de formulario y usuarios con comportamientos diferenciados.
8
9 ## Requisitos
10
11 - Node.js 18 o superior
12 - npm
13
14 ## Instalación
15
16 ```bash
17 npm install
18 npx playwright install
19 ```
20
21 ## Ejecución
22
23 ```bash
24 npm test # Suite completa
25 npm test -- --headed # Con navegador visible
26 npm test -- --debug # Modo depurador paso a paso
27 npm test -- tests/login.spec.ts # Un archivo específico
28 npm test -- --grep "carrito" # Filtran por nombre de test
29 npx playwright show-report # Abrir reporte HTML
30 ```
31
32 ## Estructura
33
34 ...
35 playwright-tests/
36 |
37 |--- pages/ # Page Objects: un archivo por pagina
38 |   |--- LoginPage.ts # Formulario de login, logout, errores
39 |   |--- InventoryPage.ts # Catalogo, carrito, menu lateral
40 |   |--- CartPage.ts # Lista de items, checkout, seguir comprando
41 |   |--- CheckoutStepOnePage.ts # Datos del cliente
42 |   |--- CheckoutStepTwoPage.ts # Resumen e importes del pedido
43 |   |--- CheckoutCompletePage.ts # Confirmacion y descarga de PDF
44 |
45 |--- tests/
46 |   |--- login.spec.ts # 9 casos de autenticacion
47 |   |--- purchase.spec.ts # 4 casos del flujo de compra
48 |   |--- playwright.config.ts
49 |
50 |--- ...
```

Se realiza ejecución de prueba “login” en playwright PATH: playwright-tests/tests/login.spec.ts

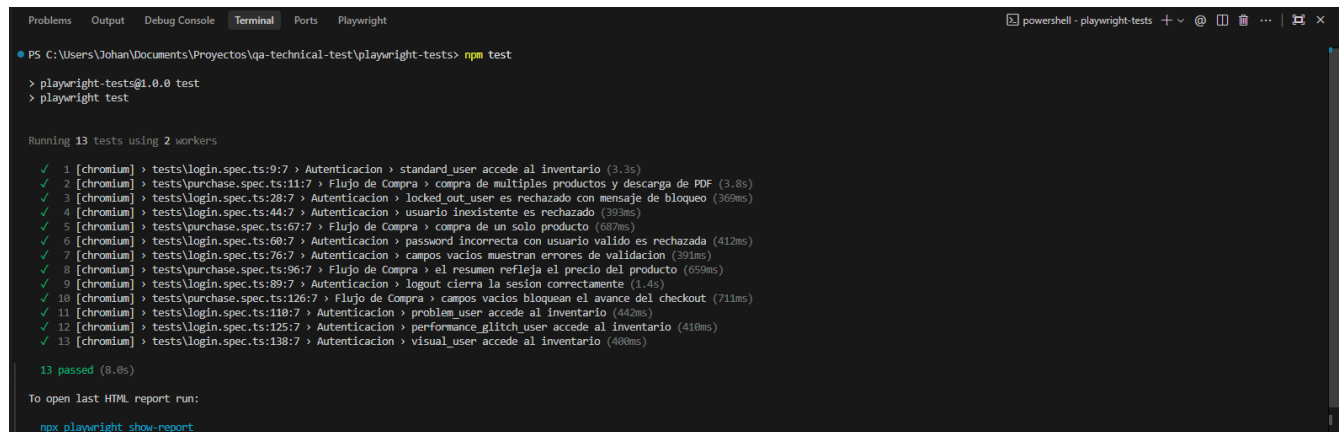


```
1 import { test, expect } from '@playwright/test';
2 import { LoginPage } from '../pages/LoginPage';
3 import { InventoryPage } from '../pages/InventoryPage';
4
5 const PASSWORD = 'secret_sauce';
6
7 test.describe('Autenticación', () => {
8
9   test('standard_user accede al inventario', async ({ page }) => {
10     const loginPage = new LoginPage(page);
11     const inventoryPage = new InventoryPage(page);
12
13     await test.step('Navegar a la pagina de login', async () => {
14       await loginPage.goto();
15     });
16
17     await test.step('Ingresar credenciales validas', async () => {
18       await loginPage.login('standard_user', PASSWORD);
19     });
20
21     await test.step('Validar redirección y contenido del inventario', async () => {
22       await expect(page).toHaveURL(/inventory\.html/);
23       await expect(inventoryPage.titleHeader).toContainText('Products');
24       await expect(inventoryPage.inventoryItems).toHaveLength(6);
25     });
26   });
27
28   test('locked_out_user es rechazado con mensaje de bloqueo', async ({ page }) => {
29     const loginPage = new LoginPage(page);
30
31     await test.step('Ingresar con locked_out_user', async () => {
32       await loginPage.goto();
33       await loginPage.login('locked_out_user', PASSWORD);
34     });
35
36     await test.step('Validar mensaje de bloqueo', async () => {
37       await expect(loginPage.errorMessage).toContainText(
38         'this user has been locked out',
39       );
40       await expect(page).not.toHaveURL(/inventory\.html/);
41     });
42   });
43
44   test('usuario inexistente es rechazado', async ({ page }) => {
45     const loginPage = new LoginPage(page);
46
47     await test.step('Ingresar con usuario inexistente', async () => {
```

Se realiza ejecución de prueba “purchase” en playwright PATH: playwright-tests/tests/purchase.spec.ts



Se evidencia la correcta ejecución de los test implementados en los suits de playwright.



Se evidencia reporte generado durante la ejecución de las pruebas.

localhost:5323

Search tests

All 13 Passed 11 Failed 0 Flaky 0 Skipped 0

Project: chromium8/10/2026, 1:25:02 p.m. Total time: 8.0s

login.spec.ts		
✓ Autenticación › standard_user accede al inventario	chromium	3.3s
login.spec.ts:9		
✓ Autenticación › locked_out_user es rechazado con mensaje de bloqueo	chromium	369ms
login.spec.ts:28		
✓ Autenticación › usuario inexistente es rechazado	chromium	393ms
login.spec.ts:44		
✓ Autenticación › password incorrecta con usuario valido es rechazada	chromium	412ms
login.spec.ts:60		
✓ Autenticación › campos vacios muestran errores de validacion	chromium	391ms
login.spec.ts:76		
✓ Autenticación › logout cierra la sesion correctamente	chromium	1.4s
login.spec.ts:89		
✓ Autenticación › problem_user accede al inventario	chromium	442ms
login.spec.ts:110		
✓ Autenticación › performance_glitch_user accede al inventario	chromium	410ms
login.spec.ts:125		
✓ Autenticación › visual_user accede al inventario	chromium	400ms
login.spec.ts:138		
purchase.spec.ts		
✓ Flujo de Compra › compra de multiples productos y descarga de PDF	chromium	3.8s
purchase.spec.ts:11		
✓ Flujo de Compra › compra de un solo producto	chromium	687ms
purchase.spec.ts:67		
✓ Flujo de Compra › el resumen refleja el precio del producto	chromium	659ms
purchase.spec.ts:96		
✓ Flujo de Compra › campos vacios bloquean el avance del checkout	chromium	711ms
purchase.spec.ts:126		